Who's Afraid of Ontologies?

Jean Bézivin

LRSG

Université de Nantes

Faculté des sciences et Techniques

2, rue de la Houssinière BP 92208

44322 Nantes cedex 3, France

tel. +33 (251) 125 813

bezivin@sciences.univ-nantes.fr

http://www.sciences.univ-nantes.fr/info/lrsg

ABSTRACT

This position paper wish to deliver the message that model engineering is becoming an essential part of software engineering and to stress the overwhelming importance of ontology-driven modeling. The notion of an ontology may sometimes convey the frightening idea of a hyper-theoretical framework with its fear of useless formalization. We wish here to convince that this is absolutely not the case and that, today, the CDIF modeler is already using ontologies without knowing it. The paper gives some concrete examples of ontologies in the object-oriented development process and concludes by illustrating some advantages of using this modeling technique.

Keywords

CDIF; ontologies; model engineering; meta-modeling ; UML; MOF;

1. Introduction

The importance of model engineering in the software development process is rapidly growing. Models are defined (constrained) by meta-models that we will call here ontologies. The CDIF modeler, will soon discover that, when dealing with "subject areas", she/he is working with ontologies in everyday life, a surprise much as Molière's Monsieur Jourdain has been "astounded to find that what daily issued from his mouth was such an important thing as *prose*".

Much work has been done on the different aspects of logic but we still miss a useful theory of models, applicable to our practical software engineering products, processes and artifacts. This need is becoming urgent. In the recent emergence of UML as an industrial standard [OMG/UML 1997], few have noticed the underlying goal of defining not a programming, an analysis or a design notation but really a "modeling" notation which is much more ambitious. As a matter of fact the state of the art is very rapidly moving in industrial circles. There is no doubt today that OMG is now centering its activities no more on one but on two different interoperability buses (Figure 1). The first one, the classical and historical software bus of the OMG is the CORBA bus, with some related recommendations like the IDL interface description language and the IIOP protocol. The second one, much more recent and still fragile, is the MOF semantic bus. The MOF (Meta-Object Facility) has taken much inspiration from CDIF [OMG/MOF 1997].

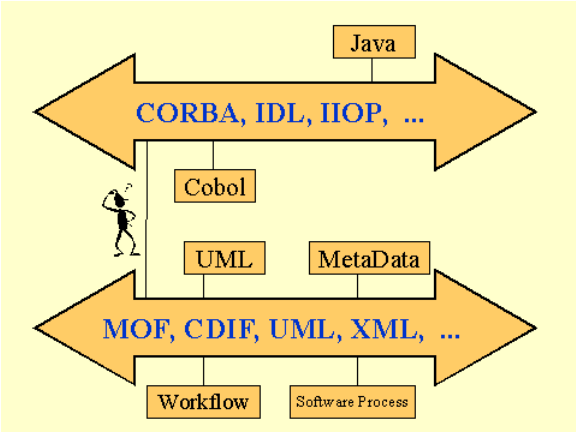

Figure 1 : The two cultures of OMG.

It is interesting to notice that in the new double vision of OMG, as illustrated in Figure 1, all efforts are being made to establish a correspondence between the software bus and the semantic bus. This means that many MOF-based recommendations have an IDL specified API. Less frequently, some CORBA-based recommendation have sometimes a UML or MOF based description.

The four-level architecture (Figure 2), very familiar to CDIFers, has been accepted as an architectural framework for models, meta-models and meta-meta-models. Initially some products proposed self-definitions, as UML with its meta-model describing UML in UML [OMG/UML 1997]. Progressively emphasis has shifted and the self defined MOF has become the reference meta-meta-model at OMG. Consequently other products are being defined in term of the MOF (Figure 3). Not only UML, but also potential extensions of UML like a considered "UML for Business Objects" notation are being defined in terms of the MOF.

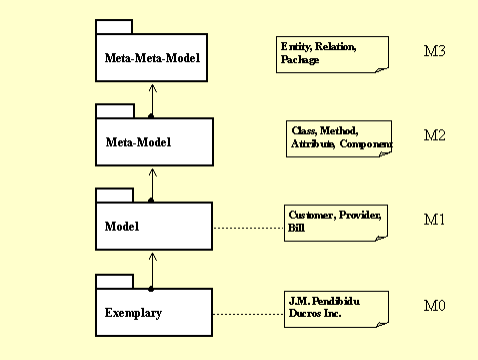

Figure 2 : The four level architecture.

Interesting ideas, like the OCL (Object Constraint Language) proposed at level M2 are being moved to level M3, recognizing the potential usefulness outside the strict domain of object analysis and design. Similarly, efforts that were envisioned for UML are being shifted up to the M3 level, like the serialization of models (SMIF). This intense activity has occurred in a very short period, but had been prepared by the long term effort of communities like the CDIF group.

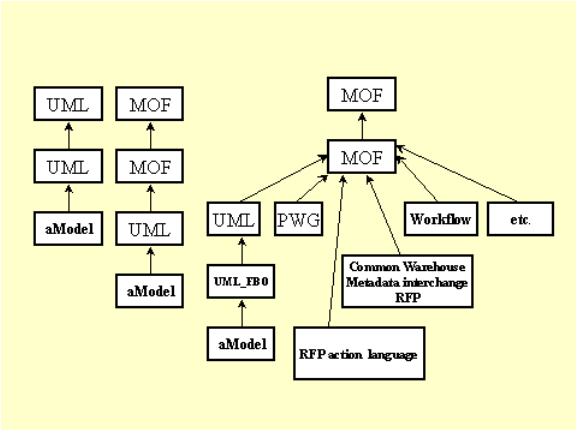

Figure 3 : The self-defined MOF.

Of course, what appears clearly now is the contradiction between Figure 2 and Figure 3. If we wish to define a given ontology called for example *UML_FBO* (UML for Business Objects), the relation between the ontological spaces *UML_FBO* and *UML* is definitively not the same than between *UML* and *MOF*. In one case it is a *basedOn(UML,MOF)* relation meaning that the *MOF* is the meta-model of *UML* whereas in the other case it is an *extends(UML_FBO,UML)* relation meaning that the ontological space *UML_FBO* is an extension (a superset) of *UML*. Since we can only add concepts and relations, the more realistic solution would

be to consider a basic ontological space called *Minimal_UML*, not including elements that are not likely to be useful for business objects (e.g. deployment views) and to consider *UML* and *UML_FBO* as extension spaces of *Minimal_UML*.

So, the example situation described in Figure 4 shows the following relations:

- *basedOn (MOF,MOF);*
- *basedOn(Minimal_UML,MOF);*
- *extends (UML_FBO,Minimal_UML);*

At the same time, other ontologies may be defined, and among them product ontologies and task ontologies. Figure 4 below shows an example of relation between the product ontology *UML_FBO* and the task (or process) ontology P_{UML_FBO}.

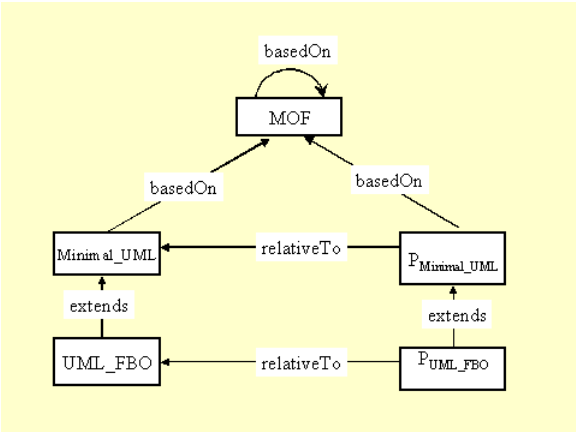

Figure 4 : Relations between product and process ontologies.

The rest of this paper is organized as follows. Section 2 will present alternative definitions on modeling and ontologies and conclude by a proposed synthesis. Section 3 will then give several examples of the use of models and ontologies in the software development process. The conclusion will endeavor to put this reflection in perspective and show which practical benefits could be reaped from using this ontological framework for software system modeling.

2. Ontologies

This section aims to clarify the sense of the term "ontology". We study several definitions proposed in the literature an finally we propose our own version, hopefully adapted to the specific needs of the software engineer.

Classical logic theory is providing few help here. We follow one thesis of Nicola Guarino [Guarino 1997] that "ontology is to possibility what logic is to truth". This is supposed to mean that ontology is concerned with the possible relation between a model and a system it is supposed to represent, in a similar way that logic is intended to deal with the possible relation between formulae, the formal value of truth and the intuitive notion of truth.

Initially the term ontology was used in philosophy with the meaning of "Theory of Existence". In the AI community, mainly following [Gruber 1992], ontology is defined as "an explicit representation of conceptualization".

Other specific communities have sometimes given restricted or slightly different definitions of the term. For the Knowledge Base community for example [Mizoguchi 1993], ontology is defined as a system of primitive vocabulary/concepts used for building artificial systems". It is legitimate, for a particular community, to adapt the notion of ontology to its specific needs and it is clear that the software engineering view will differ from the natural language processing view by emphasizing different aspects of the concepts.

In this work of clarification, there are first some terms that have to be compared and related: vocabulary, data dictionary, taxonomy, ontology and mereology.

A vocabulary is a language dependent set of explained words. It does not seek universality nor formality and it is used in a local context (for example a linguistic local context).

A data dictionary is a technical form of a vocabulary used in the specific context of computer systems development unit.

A data base schema defines aspects of a data base, such as attributes (fields), domains and parameters of the attributes.

A taxonomy is a hierarchy of concepts. In addition to the basic *isA* relation, *partOf* relation may also be used from time to time in a taxonomy.

An ontology is an explicit and precise description of concepts and relations that exists in a particular domain such as a given organization, a study field, an application area, etc. There are several levels and kinds of ontologies. The advantage of an ontology is that we are dealing with concepts and getting rid off several nasty problems usually linked to natural language vocabularies like homonymy, metonymy, polysemy, etc [Kayser 1998].

A mereology (sometimes called more precisely mereotopology), is an account of parts and wholes in a given area.

The main properties of an ontology are sharing and filtering. Sharing means that an agreement may exist between different agents, based on the acceptance of common ontologies, that they have the same understanding of a given concept. When two people are talking about an *Invoice*, they are talking about the same concept. Filtering is linked to abstraction. We are considering models of reality. These models, by definition, take into account only a part of the reality. Their usefulness is based on their ability to filter out a lot of undesirable characteristics. An ontology defines what should be extracted from a system in order to build a given model of this system (Figure 5).

Of course a model is always built for a given purpose, usually of understanding some aspects of the source system. This purpose should be clearly defined and associated to the ontology. Finally we consider only here the triad {System, Ontology, Model}, but the ontology-based extraction task will have obviously to be performed by an actor that we shall not discuss further here.

So it is becoming clear that our definition of ontology here corresponds closely to the classical definition of a meta-model as it is used in such works as [OMG/UML 1997]. An ontology contains the concepts and the relations that are relevant to a given modeling task.

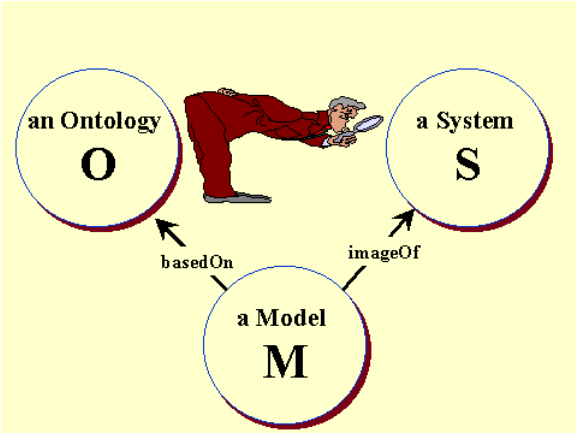

Figure 5 : An ontology as a filter.

This is the basic idea. Now an ontology may contain information of different natures. Usually there are three kinds of information (levels) inside a given ontology:

a. **Terminological**. This is the basic set of concepts and relations constituting the ontology. It is sometimes called the definition layer of the ontology.

b. **Assertional**. This part is sometimes called the axioms layer of an ontology. It is a set of assertions applying to the basic concepts and relations, e.g.:

   ~[on(x,y) ^ on(y,x)]

   bachelor (x) ≡ man (x) ^ ~married (x)

c. **Pragmatical**. This is the so-called toolbox layer. It contains a lot of pragmatical information that could not fit in a) or b). For example, if there is a concept of *Class* defined in a), then c) could contain the way to draw this concept on screen or paper, as a rounded square for example. Numerous number of pragmatical data may be found in an ontology, for example the way to serialize or to pretty print the different concepts and relations of the ontology or an API expressed in IDL It is of paramount importance that this pragmatic information could be kept packaged and close to the basic concepts it is related to.

Of course the UML practitioner has already recognized above the UML meta-model level, the OCL level and the graphical display level.

So the basic use of an ontology, is that it facilitates the separation of concerns. When dealing with a given system, you can observe and work with different models of this same system, each one characterized by a given ontology. Engineers using CDIF, UML, RM/ODP, etc. are well aware of this. Obviously when several models have been extracted from the same system with different ontologies, these systems remain related and to some extent the reverse operation may apply, namely combination of concerns. What we need for that is a clean organization of composite models, in regard to the corresponding composite meta-models.

For all this organization to work properly, what we need most is a well defined modularity concept. There may be some misunderstanding here. What we need is not the modularity concepts that have been elaborated by the programming language and compiler communities in the last 30 years. We don't need the ADA or Modula package or the namespaces with the classical import/export def/ref semantics. What we need is more close to some notion of context, but the knowledge representation community has not yet converged to a satisfactory consensual definition yet. For lack of common basis let us call space what CDIF calls a subject area, what sNets call a Universe or what UML calls a package.

A model is a space; a meta-model is a space; a meta-meta-model is also a space. This is why this notion of space is so important. If we analyze most of the meta-meta-model proposals that have been recently documented (MOFs, miniMOFs and similar constructions), they consist nearly always of the three basic notions {concept, relation, space}. The problem with most MOFs is that they add unnecessary entities at this level. Minimality is essential in a MOF and may probably be reached only by using a combination of the two relations *basedOn* and *extends*.

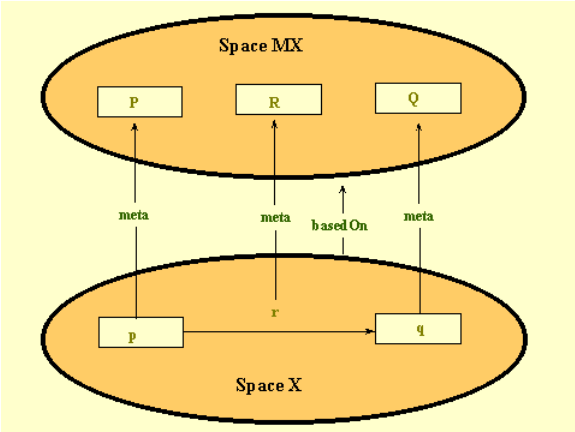

Figure 6 : The real nature of a meta-model.

In Figure 6 there are two spaces represented, a model *X* and a meta-model *MX*. For each entity present in X, there is a corresponding meta-entity present in *MX*. The relation *r(p,q)* is defined in *X*, but the concepts *P* and *Q* and the relation *R* are defined in *SX*. The relations *meta(p,P)* and *meta(q,Q)* hold. *MX* is the ontology of *X* and this corresponds to the relation *basedOn(X, MX)*. A classical error is to consider that the relation *meta(X,SX)* holds, which is obviously wrong.

3. From Object-Oriented Programming to Ontology-Driven Modeling

There are several views of the historical impact of object-oriented technology (OOT) on the overall history of computing and information systems. One of them is that OOT is entering its last decade and that its main contribution will have been to offer a smooth transition from computer level, procedure-based to the much more abstract development level, component-based culture. Another view is that OOT has triggered a much wider evolution that we may call model-oriented technology (MOT). The arrival of full-fledged MOT is not for today but may be for tomorrow or the day after. It is however of high practical interest, beyond the turbulence of present technology surface changes, to get a deeper understanding of where we are leading for. We are not announcing here the end of OOT per se but, on the contrary, its complete integration in the mainstream, leaving the front of the scene to the new model paradigm. Our thesis is that model-oriented technology (MOT) will reach a mature state well before foreseen. Many of the practical questions we are trying to solve today are really questions at the boundary between OOT and MOT. One new buzzword that we are beginning to frequently hear is ontology. Our goal in this paper has been to help defining this notion and to show how the corresponding concept is becoming central to the new emerging culture.

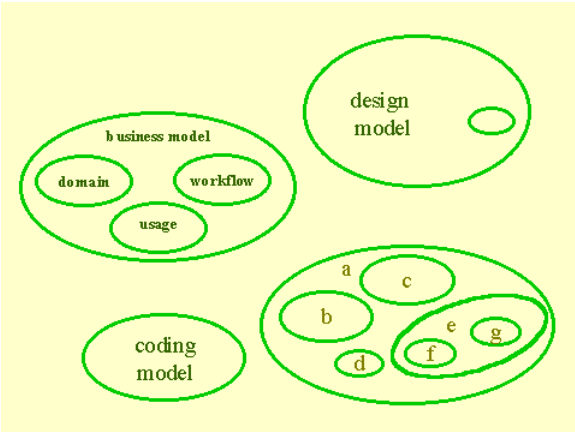

Figure 7 : Composite spaces in software engineering.

Developing a software system can be seen as a succession of model construction (Figure 7). Every model can be extracted from other models by an operation *f* called a step :

$$M_p \leftarrow f(M_i, M_j, \ldots, M_k)$$

Of course, the previous calculus may hold if we have some initial models. Some consideration can help characterizing these initial models:

a. The classical Analysis, Design, Implementation lifecycle is completely inappropriate to OOT and post-OOT development environment. Most other classical development organizations (cascade, spiral, V-shaped, fountain, etc.) also bear the seal of the procedural period and can no more be used efficiently.

b. The development task is now populated with a number of composite models, related but separated and different. Composite means that models contain elements, but may also contain other models (Figure 7).

c. Each model is based on a specific semantic, partially defined by its ontology.

d. Each model is produced, from other models, by a specific and precisely defined process. Usually these processes are cyclic concurrent processes. The processes are themselves described by task ontologies.

e. There are a restricted number of originating models, from which all other models are derived. In a simplified framework we identify three such initial models : the business model, the ressource model and the service model.

f. The design model is derived from the three initial models in a number of intermediary steps. These steps may create intermediate models.

g. In addition to the initial models, there are a number of other source models, like the reusable components model.

h. The development know-how is also present in the source models in the form of design pattern models and architectural framework models. Many new facets of component technology could be captured by associated ontologies describing the new attributes one is now taking into account.

In addition to a general theory of spaces and space representation on which we have argued here, there is the beginning of a theoretical work on the nature of the various models present in the new object-oriented development cycle. We may recognize how such a theory is badly missing when we see the time spent on discussing the real meaning of business objects and their differences with technical objects.

What is probably closest to a model-based foundation for a distinction between business space and technical space is the work of M. Jackson [Jackson 1995]. In this ongoing research, a strong distinction is made between the space of the *World* (the business domain sometimes called the universe of discourse) and the space of the *Machine* (the available computer resources, the operating systems, databases and other platform APIs). Each space is constituted of objects, processes and rules. The contribution of M. Jackson has been to show that a given computer system is at the intersection of the two spaces, i.e. that it realizes the coupling between the phenomena in the World and the phenomena in the Machine. As such it can be specified declaratively, with a high level formalism like Z for example. Alternatively the coupling may also be specified in a more operational and applied way, by using a formalism similar to Jacobson's use cases for example. This constitutes the service space in Figure 8.

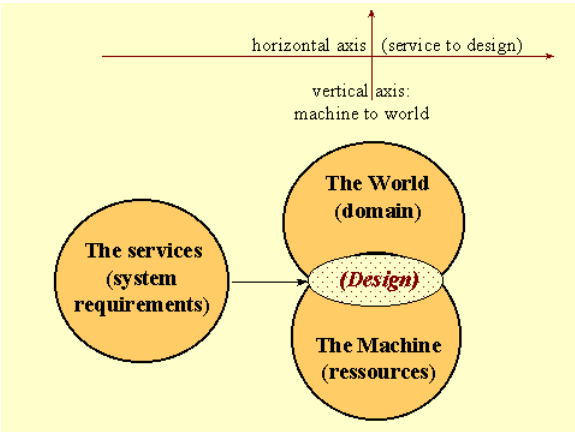

Figure 8 : Towards a theory of the four spaces.

Of course these four spaces (Service, World, Machine, Design) are composite spaces. This organization has the advantage of showing that the same elements are not needed in all ontologies. For example use case elements could be used in the service space and design patterns in the design space. At the same time obviously, the ontologies associated to the four spaces are obviously not independent.

4. Conclusions

Ontology engineering corresponds to the activity of dealing with meta-models. This activity has become much more common in the recent period. It should be based on a clear set of techniques and principles.

There is an active research community in AI ontology. We have argued here that their results could be useful to software practitioners that are more and more involved into ontology definition and alignments. Unfortunately the gap between theory and practice is still significant in this area.

The price to pay for having a sound applied theory of ontology when dealing with UML, MOF, CDIF and other practical method and model engineering is high. I tried to convince here that it is however affordable. One thing I am sure is that the price to pay for not having any underlying theory when dealing with models and meta-models will be uncomparatively larger.

As a pragmatic approach to model engineering, CDIF with its subject areas and its organizational architecture is probably the most advanced industrial platform available today. Enhanced with the XML transfer mechanisms, it seems to be the best starting point for applied research and development in this area.

Bibliography

C. Atkinson **Supporting and Applying the UML Conceptual Framework**. International Workshop <<UML>>'98, Beyond the Notation, Mulhouse, France, [June 3-4, 1998].

J. Bézivin, J. Lanneluc, R. Lemesle, **Representing Knowledge in the Object-Oriented Lifecycle** TOOLS PACIFIC'94, Melbourne, [December 1994], Prentice Hall, pp. 13-24.

J. Bézivin, **Object-Oriented Requirement Elicitation in the OSMOSIS Project** IEEE International Symposium and Workshop on Systems Engineering of Computer-Based Systems, Tucson, Arizona, pp.289-298, [6-9 March 1995].

B. Chandrasekaran, J.R. Josephson, V.R. Benjamins, **Ontology of Tasks and Methods**, 1997 AAAI Spring Symposium and the 1998 Banff Knowledge Acquisition Workshop [1998].

S. Crawley, K. Raymond, S. McBride **The Meta-Object Facility : Meta-Information Management in a CORBA World**. Tutorial Presentation, Middleware'98, The Lake District, England, [15-18 September 1998].

S. Crawley, S.Davis, J. Indulska, S. McBride, K. Raymond. **Meta Information Management**. Formal Methods for Open Object-based Distributed Systems (FMOODS) conference, Canterbury, United Kingdom, [21-23 July, 1997].

R. G. Flatscher **An Overview of the Architecture of EIA's CASE Data Interchange Format (CDIF)** http://wwwi.wu-wien.ac.at/rgf/9606mobi.html Journal of the WG 5.2, "Informationssystem Architekturen" (architecture of information systems) of the German "Gesellschaft für Informatik" (German society of informatics), SG 5.2.1, "Modellierung betrieblicher Informationssysteme" (modeling of business information systems) : "Informationssystem architekturen - Rundbrief des GI-Fachausschusses 5.2: An Overview of the Architecture of EIA's CASE Data Interchange Format (CDIF)." Volume 3, Number 1, [Summer 1996], pp. 26-30.

E. Gamma, R. Helm, R. Johnson, & J. Vlissides, **Design Patterns Elements of Reusable Object-Oriented Software** Addison Wesley Professional Computing Series, 395 p., [1994].

T. R. Gruber. **Toward principles for the design of ontologies used for knowledge sharing**. Presented at the Padua workshop on Formal Ontology, [March 1993], Available online.

M.A. Jackson, **Software requirement & Specifications A lexicon of practice, principles and prejudices** Addison Wesley, ACM Press, [1995], 228 p.

I. Jacobson, M. Christerson, P. Jonsson, G. Övergaard, **Object-Oriented Software engineering**, Addison Wesley, [1992], 524 p.)

B. Kerhervé, O. Gerbé **Models for metadata or Metamodels for Data?** Metadata'97 conference [1997].

O. Nierstrasz, D. Tsichritzis, **Object-Oriented Software Composition** Prentice Hall, [1995], 361 p.

P. Miller, **Metadata for the masses What is it, how can it help me, and how can I use it?** ARIADNE, Issue 5, ISSN: 1361-3200, [September 1996].

G.C. Murphy & D. Notkin **Reengineering with Reflexion Models : A Case Study**, IEEE, [1997]

OMG/UML **Unified Modeling Language UML Notation Guide**. AD/97-08-05, Object Management Group, Framingham, Mass., [November 1997].

OMG/MOF **Meta Object Facility (MOF) Specification**. AD/97-08-14, Object Management Group, Framingham, Mass., [September 1997].

D. Kayser **Ontologically Yours** Invited Talk, ICCS'98, Montpellier, France, LNAI #1453, M.L. Mugnier & M. Chein eds, [August 1998].

K. Raymond **Meta-Meta is Better-Better**. IFIP WG 6.1 International Working Conference on Distributed Applications and Interoperable Systems (DAIS'97) [September/October 1997].

C. Schlenoff, A. Knutilla, S. Ray **Unified Process Specification Language: Requirements for Modeling Process**, document NISTIR 5910, [September 1996].

C. Szyperski, **Component Software : Beyond Object-Oriented Programming** Addison Wesley, [1998].

M. Uschold et al **The Enterprise Ontology**, The Knowledge Engineering Review, Vol. 13, Special Issue on Putting Ontologies to Use, [1998].

C.I. Schlenoff et al., **Unified Process Specification Language: Requirements for Modeling Process**, NISTIR 5910, National Institute of Standards and Technology, Gaithersburg MD, [September 1996]. (also available at http://www.nist.gov/psl/).

M. Genesereth, R. Fikes **Knowledge Interchange Format Version 3.0 Reference Manual**, Report Logic-92-1, Stanford CA, [June 1992].